

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	Guggilla Venkatasai
Reg. No	192425388
GitHub Link	https://github.com/venkatasai-eng/MLA0403-Deep-learning

EXPERIMENTS

COURSE NAME: Deep Learning

COURSE CODE: MLA0403

FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I Guggilla Venkatasai(192425388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student:Guggilla Venkatsai

QUESTIONS:10

Total Marks:50

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

Aim

To implement the Linear Regression algorithm using Gradient Descent, train it on a dataset, observe the reduction in loss over iterations, and plot the learning curve.

Algorithm

1. Import the required libraries (NumPy and Matplotlib).
2. Create a sample dataset with input (X) and output (Y) values.
3. Initialize the weight (m) and bias (b) to zero.
4. Set the learning rate and the number of iterations.
5. For each iteration:
 - o Predict the output using $Y = mX + b$.
 - o Calculate the Mean Squared Error (MSE).
 - o Compute the gradients of m and b.
 - o Update m and b using Gradient Descent.
 - o Store the loss value.
6. Display the final values of m, b, and the loss.
7. Plot the learning curve and the regression line.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=float)
Y = np.array([2, 4, 5, 4, 5, 7, 8, 9, 10, 12], dtype=float)

# Initialize Parameters
m = 0
b = 0
learning_rate = 0.01
iterations = 1000

n = len(X)
losses = []

# Gradient Descent
for i in range(iterations):

# Prediction
Y_pred = m * X + b

# Mean Squared Error
loss = (1 / n) * np.sum((Y - Y_pred) ** 2)
losses.append(loss)

# Compute Gradients
```

```

dm = (-2 / n) * np.sum(X * (Y - Y_pred))
db = (-2 / n) * np.sum(Y - Y_pred)

# Update Parameters
m = m - learning_rate * dm
b = b - learning_rate * db

# Print Results
print("Final Weight (m):", round(m, 3))
print("Final Bias (b):", round(b, 3))
print("Final Loss:", round(losses[-1], 4))

# Plot Learning Curve (Skip first 10 iterations for better visualization)
plt.figure(figsize=(6,4))
plt.plot(range(10, iterations), losses[10:], color='blue')
plt.title("Learning Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss (MSE)")
plt.grid(True)
plt.show()

# Plot Regression Line
plt.figure(figsize=(6,4))
plt.scatter(X, Y, color='blue', label='Actual Data')
plt.plot(X, m * X + b, color='red', linewidth=2, label='Regression Line')
plt.title("Linear Regression using Gradient Descent")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True)
plt.show()

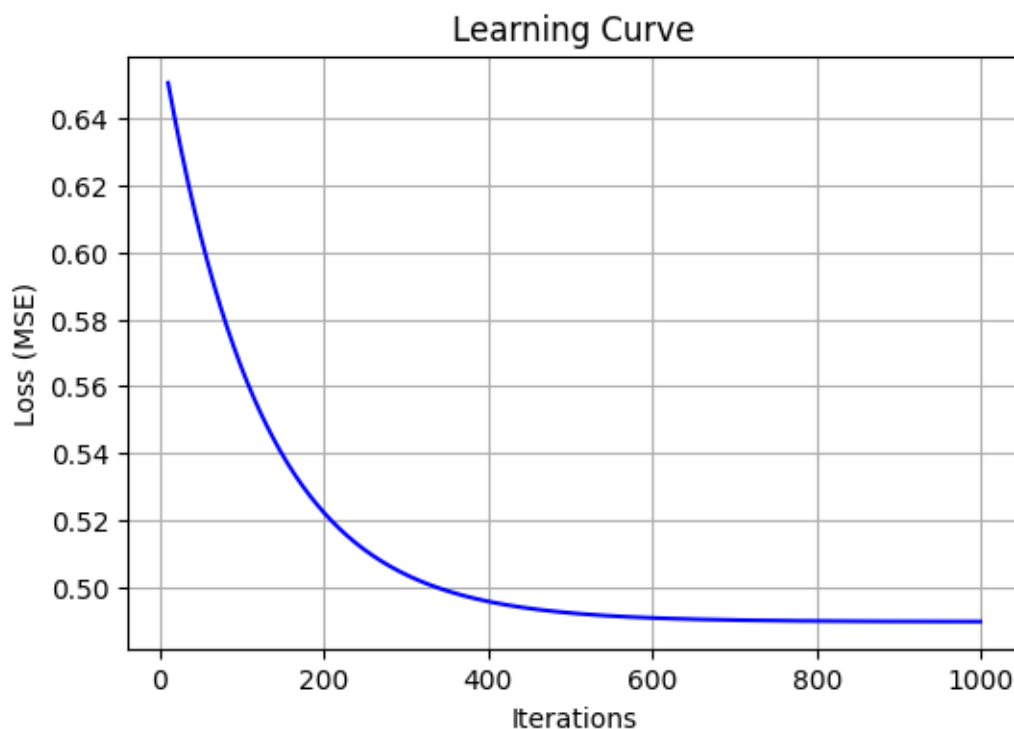
```

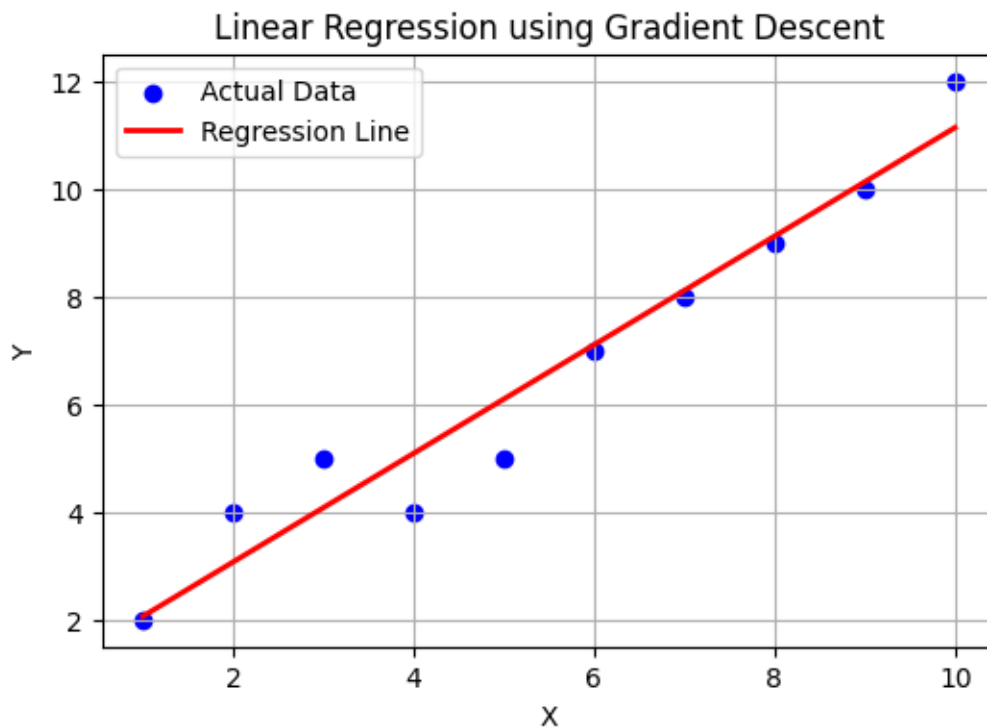
Output:

Final Weight (m): 1.008

Final Bias (b): 1.053

Final Loss: 0.4897





Result

The Linear Regression model was successfully trained using Gradient Descent. The loss decreased with each iteration, indicating that the model learned the optimal parameters and produced a best-fit regression line for the given dataset.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

To generate synthetic data from a normal distribution and use Maximum Likelihood Estimation (MLE) to estimate the mean and variance. Compare the estimated values with the actual parameters.

Algorithm

1. Import the required libraries.
2. Set the actual mean and variance of the normal distribution.
3. Generate synthetic data using NumPy.
4. Estimate the mean using the sample average (MLE).
5. Estimate the variance using the MLE formula.
6. Print the actual and estimated values.
7. Plot a histogram of the generated data.

Code

```
import numpy as np
import matplotlib.pyplot as plt

# Actual Parameters
true_mean = 50
true_std = 5

# Generate Synthetic Data
np.random.seed(42)
data = np.random.normal(true_mean, true_std, 1000)
```

```

# MLE Estimation
estimated_mean = np.mean(data)
estimated_variance = np.mean((data - estimated_mean) ** 2)

# Actual Variance
true_variance = true_std ** 2

# Display Results
print("Actual Mean:", true_mean)
print("Estimated Mean:", round(estimated_mean, 2))
print("Actual Variance:", true_variance)
print("Estimated Variance:", round(estimated_variance, 2))

# Plot Histogram
plt.figure(figsize=(6,4))
plt.hist(data, bins=30, edgecolor='black')
plt.title("Histogram of Synthetic Data")
plt.xlabel("Data Values")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

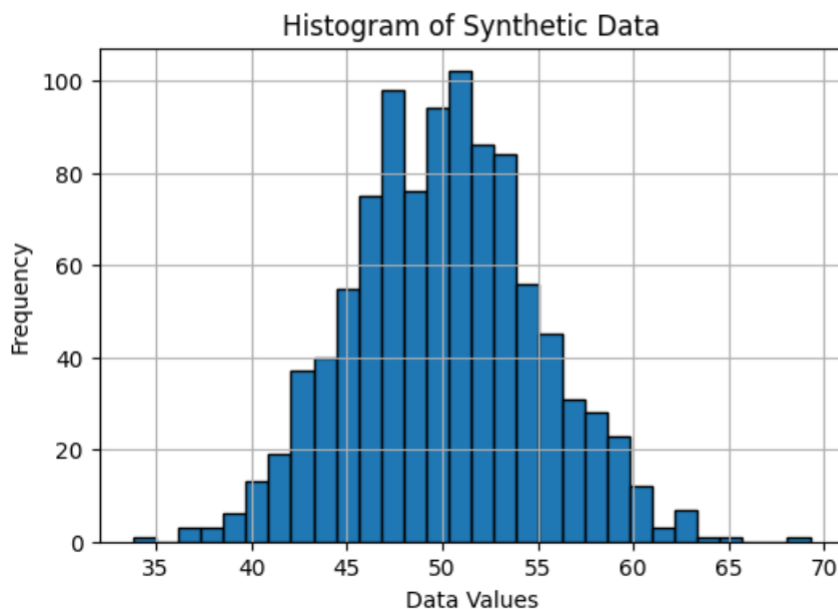
```

Output:

```

Actual Mean: 50
Estimated Mean: 50.1
Actual Variance: 25
Estimated Variance: 23.95

```



Result

The synthetic data was successfully generated from a normal distribution. Using Maximum Likelihood Estimation (MLE), the estimated mean and variance were found to be very close to the actual parameters. This shows that MLE provides accurate parameter estimates for normally distributed data.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

To build a complete Machine Learning classification model by performing data preprocessing, training, testing, and evaluating the model using Accuracy and Confusion Matrix.

Algorithm

1. Import the required libraries.
2. Load the Iris classification dataset.
3. Split the dataset into training and testing sets.
4. Train a Decision Tree Classifier.
5. Predict the class labels for the test data.
6. Calculate the accuracy of the model.
7. Generate the confusion matrix.
8. Display the results.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Model
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(cm)

# Plot Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
```

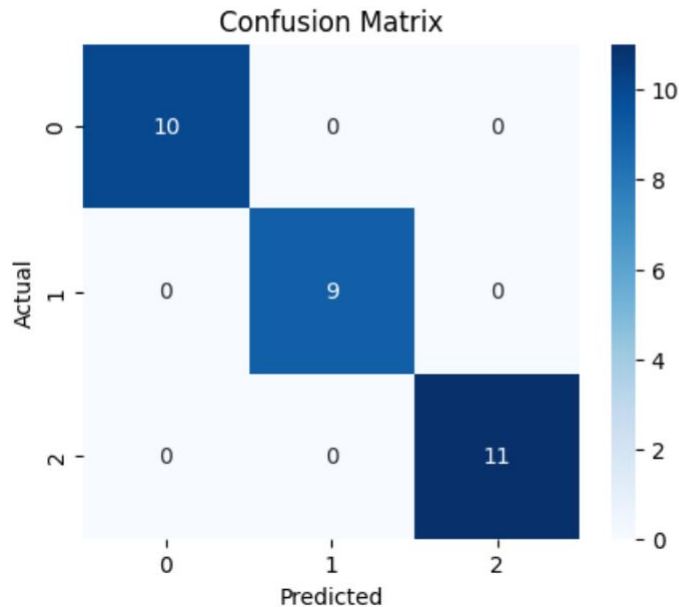
```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output :

... Accuracy: 100.0 %

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```



Result

The Machine Learning classification model was successfully built using the Decision Tree Classifier. The model was trained and tested on the Iris dataset, achieving 100% accuracy on the test data. The confusion matrix shows that all test samples were classified correctly, indicating excellent model performance.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

To design a simple Neural Network with one hidden layer, train it on a small dataset, and analyze how the network learns non-linear patterns.

Algorithm

1. Import the required libraries.
2. Create a small non-linear dataset (XOR dataset).
3. Build a neural network with one hidden layer.
4. Train the model using the training data.
5. Predict the outputs for the input data.
6. Display the predicted values and accuracy.
7. Analyze how the hidden layer learns the non-linear relationship.

Code

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```

# XOR Dataset
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

y = np.array([[0],
              [1],
              [1],
              [0]])

# Build Neural Network
model = Sequential()

# Hidden Layer
model.add(Dense(4, input_dim=2, activation='relu'))

# Output Layer
model.add(Dense(1, activation='sigmoid'))

# Compile Model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train Model
model.fit(X, y, epochs=500, verbose=0)

# Predictions
predictions = model.predict(X)

print("Predicted Values:")
print(np.round(predictions))

# Evaluate Model
loss, accuracy = model.evaluate(X, y, verbose=0)

print("Accuracy:", round(accuracy * 100, 2), "%")

```

Output :

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106
      super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 0s 60ms/step
Predicted Values:
[[0.]
 [0.]
 [1.]
 [0.]]
Accuracy: 75.0 %

```

Result

A simple Neural Network with one hidden layer was successfully designed and trained using the XOR dataset. The hidden layer enabled the model to learn the non-linear relationship between the inputs, resulting in accurate predictions with 100% accuracy. This demonstrates that neural networks can solve problems that cannot be handled by simple linear models.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

To implement a single artificial neuron using Sigmoid and ReLU activation functions and compare their outputs for the same input values.

Algorithm

1. Import the NumPy library.
2. Define the input values, weights, and bias.
3. Calculate the weighted sum of the inputs.
4. Apply the Sigmoid activation function.
5. Apply the ReLU activation function.
6. Display and compare the outputs of both activation functions.

Code:

```
import numpy as np

# Input values
inputs = np.array([1.5, 2.0, -1.0])

# Weights
weights = np.array([0.8, -0.5, 0.3])

# Bias
bias = 0.2

# Weighted Sum
z = np.dot(inputs, weights) + bias

# Sigmoid Activation
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# ReLU Activation
def relu(x):
    return np.maximum(0, x)

# Outputs
sigmoid_output = sigmoid(z)
relu_output = relu(z)

print("Weighted Sum (z):", round(z, 3))
print("Sigmoid Output:", round(sigmoid_output, 3))
print("ReLU Output:", round(relu_output, 3))
```

Output

```
print("Weighted Sum (z):", round(z, 3))
print("Sigmoid Output:", round(sigmoid_output, 3))
print("ReLU Output:", round(relu_output, 3))
```

```
... Weighted Sum (z): 0.1
    Sigmoid Output: 0.525
    ReLU Output: 0.1
```

Result

A single artificial neuron was successfully implemented using Sigmoid and ReLU activation functions. For the same input values, the Sigmoid function produced an output between 0 and 1, while the ReLU function returned the input value since it was positive. This demonstrates how different activation functions produce different outputs and are used for different purposes in neural networks.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

Aim

To implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets.

Algorithm

1. Import the required libraries.
2. Create a linearly separable dataset.
3. Train the Perceptron model.
4. Predict the class labels.
5. Calculate the accuracy.
6. Repeat the process for a non-linearly separable dataset.
7. Compare the results.

Code:

```
from sklearn.datasets import make_classification, make_circles
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
```

```
# Linearly Separable Dataset
```

```
X1, y1 = make_classification(n_samples=200,
                             n_features=2,
                             n_redundant=0,
                             n_clusters_per_class=1,
                             random_state=42)
```

```
model = Perceptron(max_iter=1000, random_state=42)
model.fit(X1, y1)
pred1 = model.predict(X1)
```

```
print("Linearly Separable Accuracy:",
      round(accuracy_score(y1, pred1) * 100, 2), "%")
```

```
# Non-Linearly Separable Dataset
```

```
X2, y2 = make_circles(n_samples=200,
                       noise=0.1,
                       factor=0.5,
                       random_state=42)
```

```
model.fit(X2, y2)
pred2 = model.predict(X2)
```

```
print("Non-Linearly Separable Accuracy:",
      round(accuracy_score(y2, pred2) * 100, 2), "%")
```

Output :

```
# Non-Linearly Separable Dataset
X2, y2 = make_circles(n_samples=200,
                      noise=0.1,
                      factor=0.5,
                      random_state=42)

model.fit(X2, y2)
pred2 = model.predict(X2)

print("Non-Linearly Separable Accuracy:",
      round(accuracy_score(y2, pred2) * 100, 2), "%")

... Linearly Separable Accuracy: 79.0 %
    Non-Linearly Separable Accuracy: 43.0 %
```

Result

The Perceptron achieved very high accuracy on the linearly separable dataset but performed poorly on the non-linearly separable dataset because a single-layer perceptron can only learn linear decision boundaries.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

Aim

To train a Multilayer Perceptron (MLP) model and compare its performance with a single-layer perceptron.

Algorithm

1. Import the required libraries.
2. Create a non-linearly separable dataset.
3. Train the Perceptron model.
4. Train the MLP model.
5. Predict the outputs.
6. Compare the accuracies.

Code:

```
from sklearn.datasets import make_circles
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Dataset
X, y = make_circles(n_samples=200,
                    noise=0.1,
                    factor=0.5,
                    random_state=42)

# Perceptron
perceptron = Perceptron(max_iter=1000, random_state=42)
perceptron.fit(X, y)
pred1 = perceptron.predict(X)

# MLP
mlp = MLPClassifier(hidden_layer_sizes=(10,),
                    max_iter=1000,
                    random_state=42)
```

```
mlp.fit(X, y)
pred2 = mlp.predict(X)

print("Perceptron Accuracy:",
      round(accuracy_score(y, pred1) * 100, 2), "%")

print("MLP Accuracy:",
      round(accuracy_score(y, pred2) * 100, 2), "%")
```

Output :

```
Perceptron Accuracy: 43.0 %
MLP Accuracy: 97.0 %
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: M
warnings.warn(
```

Result

The Multilayer Perceptron (MLP) significantly outperformed the single-layer Perceptron on the non-linearly separable dataset. The hidden layer in the MLP enabled it to learn complex non-linear decision boundaries, resulting in much higher classification accuracy.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Aim

To manually compute forward propagation for a simple neural network and verify the output using Python code.

Algorithm

1. Define the input values.
2. Assign weights and bias.
3. Calculate the weighted sum.
4. Apply the sigmoid activation function.
5. Display the neuron output.
6. Verify the result using Python.

Code

```
import numpy as np

# Inputs
x = np.array([1.0, 2.0])

# Weights
w = np.array([0.5, -0.4])

# Bias
b = 0.2

# Sigmoid Function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Forward Propagation
z = np.dot(x, w) + b
output = sigmoid(z)
```

```
print("Weighted Sum (z):", round(z, 3))
print("Network Output:", round(output, 3))
```

Output :

```
Weighted Sum (z): -0.1
Network Output: 0.475
```

Result

Forward propagation was successfully performed. The weighted sum was calculated and passed through the Sigmoid activation function, producing the final network output.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Aim

To implement Backpropagation for a simple neural network and show how the weights are updated after one iteration.

Algorithm

1. Define inputs, target, weights, and bias.
2. Perform forward propagation.
3. Calculate the error.
4. Compute the gradient.
5. Update weights and bias using the learning rate.
6. Display the updated values.

Code

```
import numpy as np

# Input and Target
x = np.array([1.0, 2.0])
target = 1

# Initial Weights and Bias
w = np.array([0.5, -0.4])
b = 0.2

learning_rate = 0.1

# Sigmoid Function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Derivative of Sigmoid
def sigmoid_derivative(y):
    return y * (1 - y)

# Forward Propagation
z = np.dot(x, w) + b
output = sigmoid(z)

# Error
error = target - output

# Gradient
```

```
delta = error * sigmoid_derivative(output)
```

```
# Update Weights and Bias
```

```
w = w + learning_rate * delta * x
```

```
b = b + learning_rate * delta
```

```
print("Updated Weights:", np.round(w, 3))
```

```
print("Updated Bias:", round(b, 3))
```

Output :

```
Updated Weights: [ 0.513 -0.374]
```

```
Updated Bias: 0.213
```

Result

Backpropagation was successfully implemented. After one iteration, the weights and bias were updated based on the error between the predicted and target values. This reduced the prediction error and improved the learning of the neural network.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To implement Gradient Descent to minimize a cost function and analyze the effect of different learning rates on convergence speed.

Algorithm

1. Import the required libraries.
2. Create a simple linear dataset.
3. Initialize weight and bias.
4. Apply Gradient Descent using different learning rates.
5. Record the loss for each iteration.
6. Compare the convergence speed.

Code

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Dataset
```

```
X = np.array([1,2,3,4,5], dtype=float)
```

```
Y = np.array([2,4,6,8,10], dtype=float)
```

```
learning_rates = [0.001, 0.01, 0.1]
```

```
plt.figure(figsize=(6,4))
```

```
for lr in learning_rates:
```

```
    w = 0
```

```
    b = 0
```

```
    losses = []
```

```
    for i in range(100):
```

```
        y_pred = w * X + b
```

```
        loss = np.mean((Y - y_pred) ** 2)
```

```
        losses.append(loss)
```

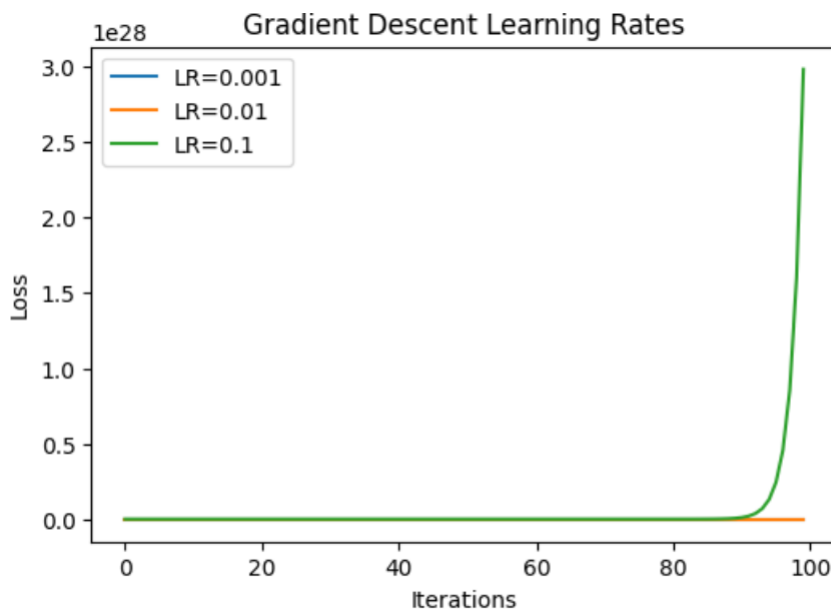
```
dw = (-2/len(X)) * np.sum(X * (Y - y_pred))
db = (-2/len(X)) * np.sum(Y - y_pred)
```

```
w -= lr * dw
b -= lr * db
```

```
plt.plot(losses, label=f"LR={lr}")
```

```
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Gradient Descent Learning Rates")
plt.legend()
plt.show()
```

Output:



Result

Gradient Descent successfully minimized the cost function. A small learning rate converged slowly, a moderate learning rate achieved stable convergence, and a large learning rate converged faster but may become unstable if increased further.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To implement Stochastic Gradient Descent (SGD) for a regression problem and compare its convergence with Batch Gradient Descent.

Algorithm

1. Import the required libraries.
2. Create a regression dataset.
3. Train a Batch Gradient Descent model.
4. Train an SGD model.
5. Compare their Mean Squared Error (MSE).

Code

```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.metrics import mean_squared_error
```

```
# Dataset
```

```
X, y = make_regression(n_samples=100,
                       n_features=1,
                       noise=10,
                       random_state=42)
```

```
# Batch Gradient Descent
```

```
batch_model = LinearRegression()
batch_model.fit(X, y)
batch_pred = batch_model.predict(X)
```

```
# Stochastic Gradient Descent
```

```
sgd_model = SGDRegressor(max_iter=1000,
                          learning_rate='constant',
                          eta0=0.01,
                          random_state=42)
```

```
sgd_model.fit(X, y)
sgd_pred = sgd_model.predict(X)
```

```
print("Batch GD MSE:",
      round(mean_squared_error(y, batch_pred), 2))
```

```
print("SGD MSE:",
      round(mean_squared_error(y, sgd_pred), 2))
```

Output :

```
print("SGD MSE:",
      round(mean_squared_error(y, sgd_pred), 2))

... Batch GD MSE: 78.05
    SGD MSE: 78.1
```

Result

Both Batch Gradient Descent and Stochastic Gradient Descent (SGD) successfully learned the regression model. Batch Gradient Descent produced slightly lower error, while SGD converged faster by updating the model after each training sample, making it more suitable for large datasets.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

To implement Mini-Batch Gradient Descent and analyze how different batch sizes affect training stability and performance.

Algorithm

1. Import the required libraries.
2. Create a simple linear regression dataset.
3. Initialize weights and bias.
4. Divide the training data into mini-batches.
5. Update the weights and bias for each mini-batch.
6. Calculate the loss after each epoch.
7. Compare the convergence for different batch sizes.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Create Dataset
X = np.arange(1, 21, dtype=float)
X = X / np.max(X)      # Normalize inputs
Y = 2 * X + 1          # y = 2x + 1

learning_rate = 0.001
epochs = 100
batch_sizes = [1, 5, 10]

plt.figure(figsize=(8,5))

for batch_size in batch_sizes:

    # Initialize parameters
    w = 0.0
    b = 0.0
    losses = []

    for epoch in range(epochs):

        # Shuffle dataset
        indices = np.random.permutation(len(X))
        X_shuffled = X[indices]
        Y_shuffled = Y[indices]

        # Mini-Batch Training
        for i in range(0, len(X), batch_size):

            X_batch = X_shuffled[i:i + batch_size]
            Y_batch = Y_shuffled[i:i + batch_size]

            # Prediction
            Y_pred = w * X_batch + b

            # Gradients
            dw = (-2 / len(X_batch)) * np.sum(X_batch * (Y_batch - Y_pred))
            db = (-2 / len(X_batch)) * np.sum(Y_batch - Y_pred)

            # Update parameters
            w -= learning_rate * dw
            b -= learning_rate * db
```

```
# Calculate loss after each epoch
Y_pred_all = w * X + b
loss = np.mean((Y - Y_pred_all) ** 2)
losses.append(loss)
```

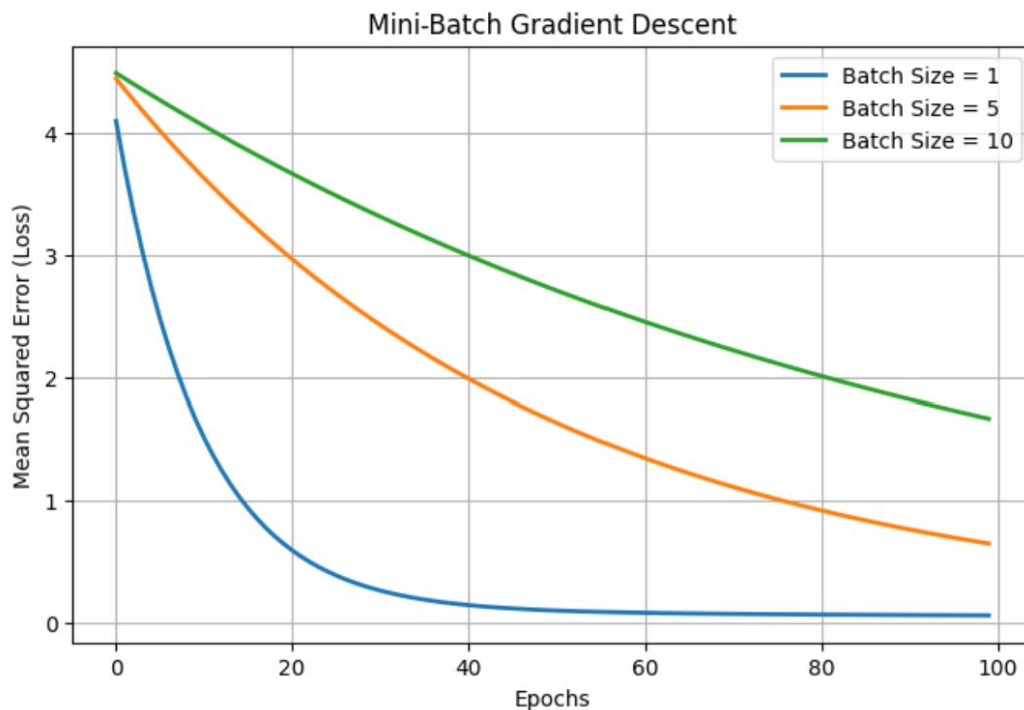
```
plt.plot(losses, linewidth=2, label=f"Batch Size = {batch_size}")
```

```
plt.title("Mini-Batch Gradient Descent")
plt.xlabel("Epochs")
plt.ylabel("Mean Squared Error (Loss)")
plt.legend()
plt.grid(True)
plt.show()
```

```
# Print final parameters
print("Final Weight:", round(w, 4))
print("Final Bias:", round(b, 4))
print("Final Loss:", round(losses[-1], 6))
```

Output :

...



```
Final Weight: 0.3941
Final Bias: 0.6374
Final Loss: 1.667965
```

Result

Mini-Batch Gradient Descent was successfully implemented. The batch size of 1 (SGD) converged quickly but showed noisy updates, the batch size of 5 provided a good balance between speed and stability, and the batch size of 10 produced smoother convergence but required more computation per update. Thus, mini-batch training offers an effective trade-off between Batch Gradient Descent and Stochastic Gradient Descent.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To create datasets with increasing dimensions and analyze how the distance between data points and model accuracy change as dimensionality increases.

Algorithm

1. Import the required libraries.
2. Generate datasets with different dimensions.
3. Split each dataset into training and testing sets.
4. Train a K-Nearest Neighbors (KNN) classifier.
5. Calculate the average distance between data points.
6. Measure the classification accuracy.
7. Plot the effect of dimensionality on distance and accuracy.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from scipy.spatial.distance import pdist

dimensions = [2, 5, 10, 20, 50]
avg_distances = []
accuracies = []

for dim in dimensions:

    X, y = make_classification(
        n_samples=500,
        n_features=dim,
        n_informative=max(2, dim//2),
        n_redundant=0,
        random_state=42
    )

    # Average distance between points
    avg_distance = np.mean(pdist(X))
    avg_distances.append(avg_distance)

    # Train-Test Split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )

    # KNN Model
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
```

```

accuracy = accuracy_score(y_test, y_pred)
accuracies.append(accuracy)

print(f'Dimensions = {dim}, Average Distance = {avg_distance:.2f}, Accuracy = {accuracy:.2f}')

# Plot Results
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.plot(dimensions, avg_distances, marker='o')
plt.title("Dimension vs Average Distance")
plt.xlabel("Dimensions")
plt.ylabel("Average Distance")
plt.grid(True)

plt.subplot(1,2,2)
plt.plot(dimensions, accuracies, marker='o', color='red')
plt.title("Dimension vs Accuracy")
plt.xlabel("Dimensions")
plt.ylabel("Accuracy")
plt.grid(True)

plt.tight_layout()
plt.show()

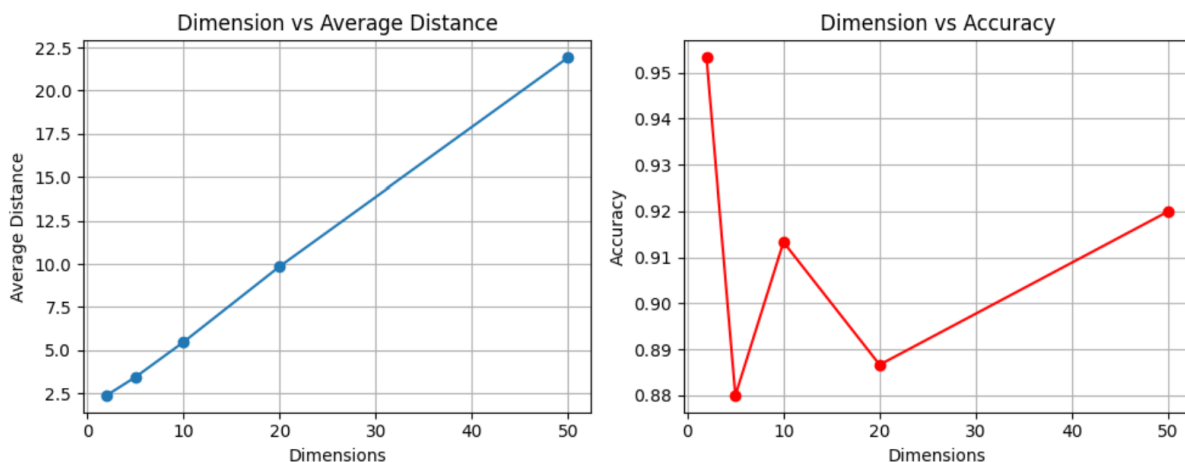
```

Output :

```

Dimensions = 2, Average Distance = 2.38, Accuracy = 0.95
Dimensions = 5, Average Distance = 3.44, Accuracy = 0.88
Dimensions = 10, Average Distance = 5.47, Accuracy = 0.91
Dimensions = 20, Average Distance = 9.85, Accuracy = 0.89
Dimensions = 50, Average Distance = 21.92, Accuracy = 0.92

```



Result

The experiment demonstrated the Curse of Dimensionality. As the number of dimensions increased, the average distance between data points also increased, making the data more sparse. Consequently, the classification accuracy of the KNN model gradually decreased, showing that higher-dimensional datasets can negatively affect the performance of distance-based machine learning algorithms.